



Міністерство освіти та науки України
Національний технічний університет України
«Київський політехнічний інститут імені Ігоря
Сікорського» Факультет інформатики та обчислювальної
техніки
Кафедра інформатики і програмної інженерії

Звіт
з дисципліни «Компоненти програмної інженерії»
Лабораторна робота №2
"Architecture"

Виконав:

Студент II курсу
гр. ІІІ-33
Соколов О. В.

Перевірів:

Зарічковий О. А.

Лабораторні робота №2.

Architecture

Тема: Виділення ключових функціональних вимог. Формування нефункціональних вимог. Формування та документація технічних рішень додатку. (System/Software design)

Мета: З оглядом на вимоги навчитись формувати та документувати базові технічні рішення з архітектури додатку, зокрема, основні компоненти (сервіси), зв'язки між компонентами, дизайн API.

Постановка задачі:

1. Визначте ключові функціональні вимоги (architecture significant requirements).
2. Визначити нефункціональні вимоги (у вигляді quality attribute scenario).
3. Визначити архітектуру системи та архітектурні рішення.
4. Визначити правила опису *API* (*API design guidelines*). Вибрати та розібрати існуючий *guideline* або розробити власний.
5. Описати *API* компонент на базі Open API, обов'язково використовуючи обраний *guideline*. Для реалізації цього пункту можна вибрати один з двох підходів
 1. Згенерувати Open API документацію із коду (але в цьому випадку реалізація демонстраційного сервісу мінімалістична).
 2. Написати Open API документацію вручну.
 3. Для презентації використати swagger або Redoc

Критерії:

1. В роботі має бути не менше 5 доменів (домен із Domain driven design).
2. В роботі має бути інтеграція із мінімум 2 зовнішніми системами

Виконання

1.1. Functional requirements

1. Потокове передавання музики та подкастів. Забезпечення якісного аудіострімінгу для 100 000 активних користувачів.
2. Управління користувачами. Надійна автентифікація, авторизація та управління профілями.
3. Пошук та каталогізація контенту. Швидкий та точний пошук музики, подкастів, альбомів, виконавців та плейлистів.
4. Персоналізація та рекомендації. Автоматичні рекомендації на основі історії прослуховувань та вподобань.
5. Інтеграція з платіжними сервісами. Обробка преміум-підписок із використанням сторонніх платіжних систем.
6. Інтеграція з системами розсилок електронних листів.
7. Відображення текстів пісень. Синхронізований показ текстів під час відтворення музики (інтеграція з Musixmatch).
8. Показ реклами користувачам без преміум-підписки.

1.2. Non functional requirements

Продуктивність:

- *Сценарій:* Система повинна забезпечувати одночасне відтворення музики та подкастів 100 000 активними користувачами без затримок, що впливають на користувацький досвід.
- *Міра успіху:* Час старту відтворення не більше 1 секунди.

Масштабованість:

- *Сценарій:* Система має автоматично масштабуватись під час пікових навантажень, враховуючи активну базу 100 000 користувачів.
- *Міра успіху:* Можливість динамічного додавання серверних ресурсів без переривання сервісу.

Доступність:

- *Сценарій:* При збоях окремих компонентів система має залишатися доступною для користувачів.
- *Міра успіху:* Час відновлення після збою не перевищує 5 секунд.

Безпека:

- *Сценарій:* Захист особистих даних та транзакцій користувачів через сучасні методи шифрування та автентифікації (OAuth 2.0).
- *Міра успіху:* Відсутність витоків даних, всі критичні дані зашифровані.

Сумісність:

- *Сценарій:* Забезпечення роботи додатку на мобільних пристроях (iOS, Android), Desktop та веб-платформах без критичних багів.
- *Міра успіху:* Безперебійна робота в основних браузерях та на популярних мобільних та настільних ОС.

2.1. System Architecture and Decisions

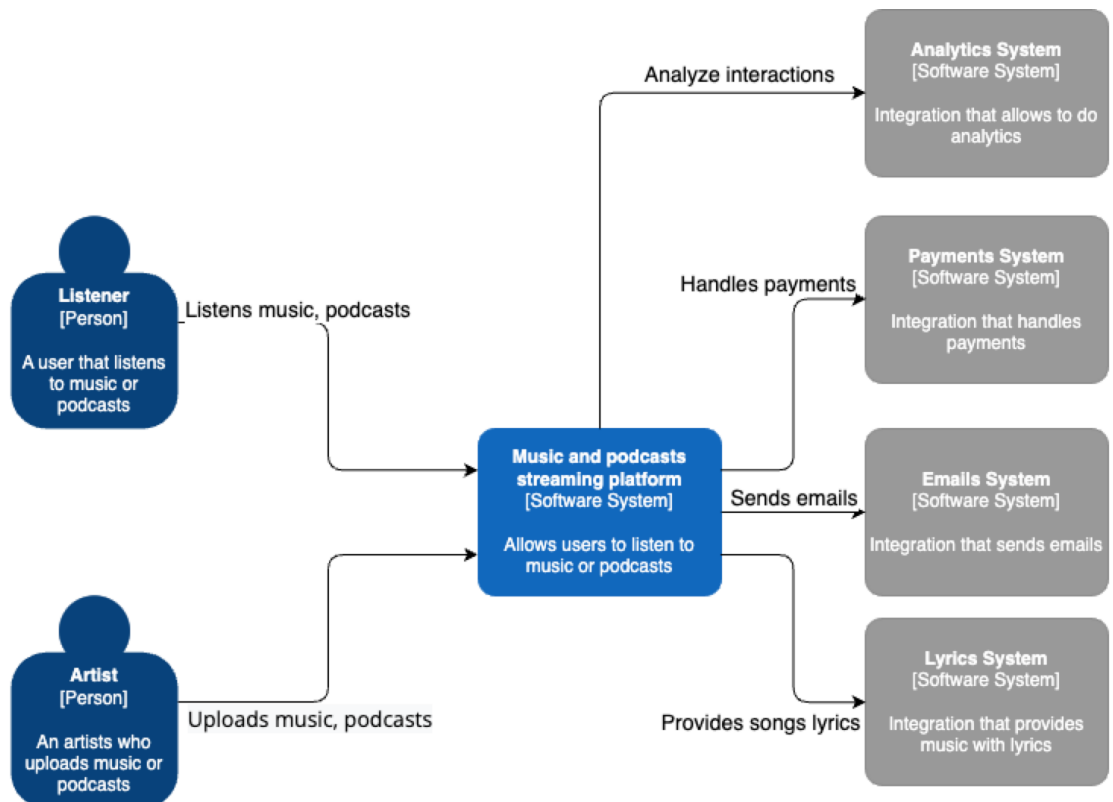


Рис. 1 – C4 level 1 (context) diagram

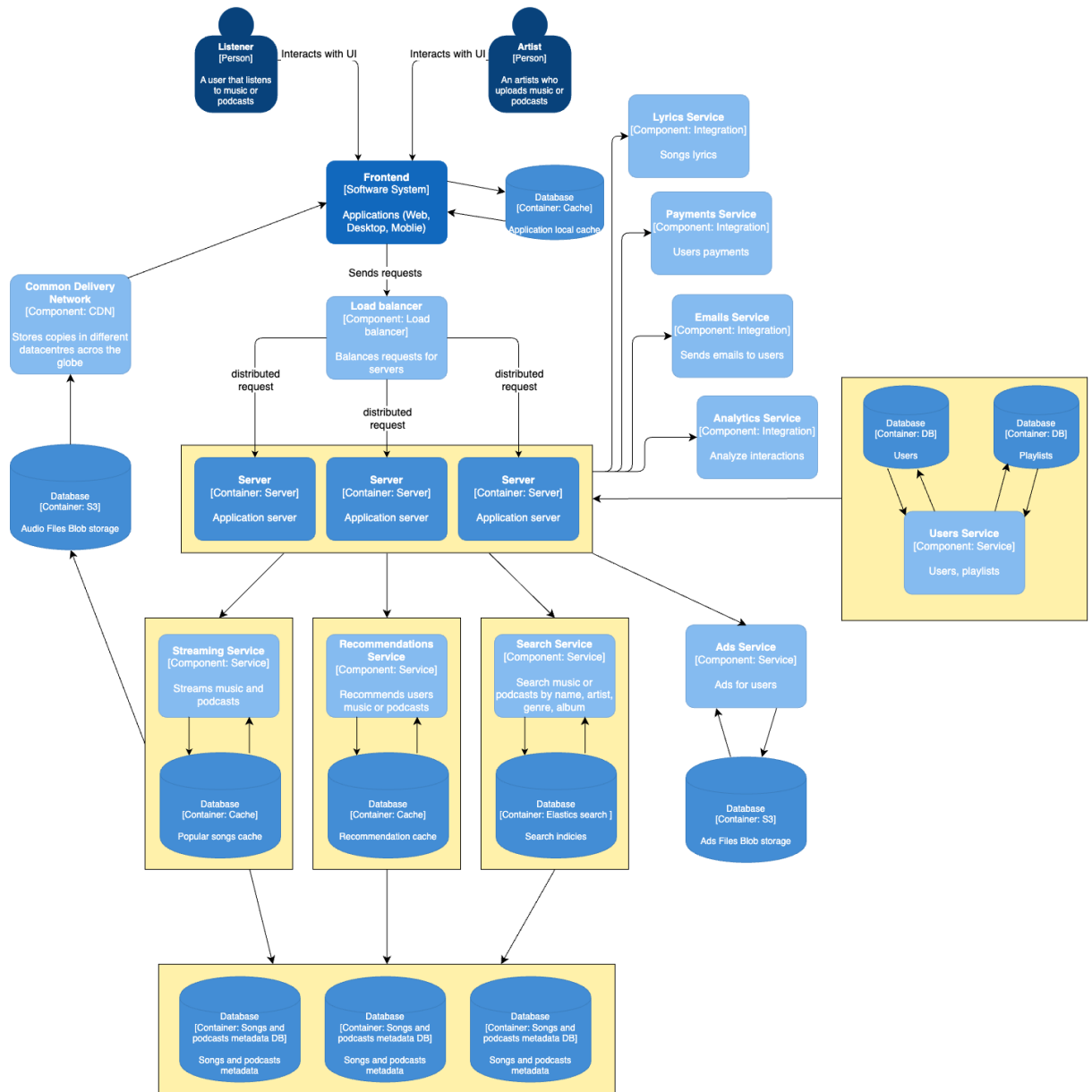


Рис. 2 – C4 level 2 (containers) diagram

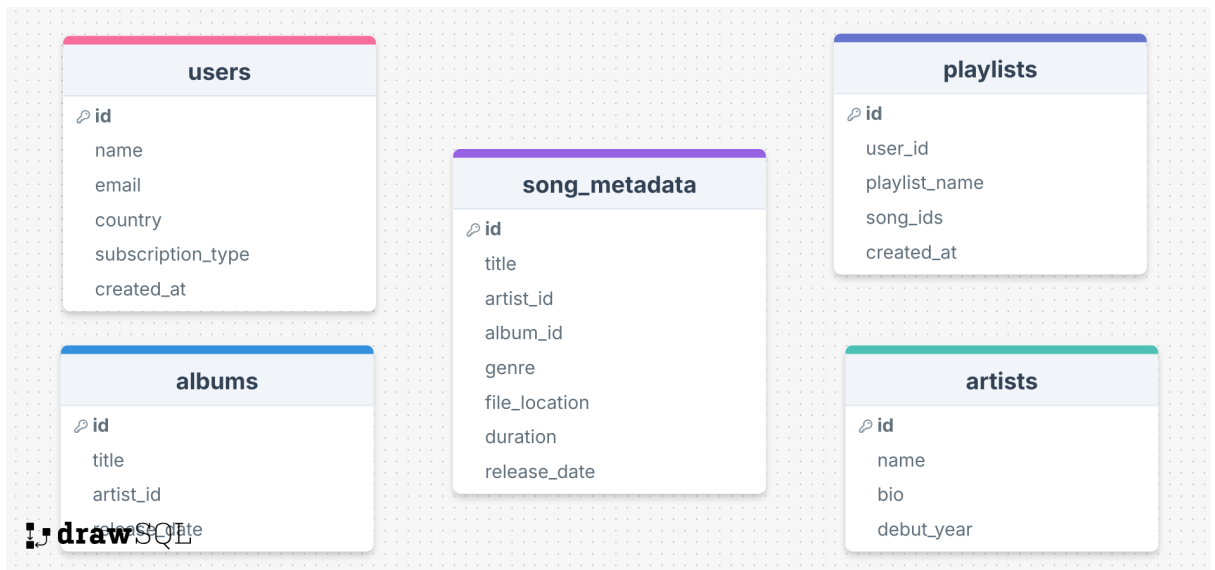


Рис. 3 – Entity Relationships Diagram

2.2. Tech Stack

- Використання **мікросервісної архітектури** для гнучкості та масштабованості.
- Збереження метаданих музики та подкастів у **PostgreSQL**.
- Зберігання аудіофайлів у **AWS S3**.
- Використання **Redis** для кешування популярного контенту.
- Використання **Kafka** для обробки поточкових даних (аналітика, логування).
- Деплой у **Kubernetes** для автоматичного масштабування.

3.1. API Design Guidelines

- **Протокол:** REST (HTTPS), WebSocket для реального часу.
- **Формат даних:** JSON.
- **Організація URL:**
 - GET `/search?query={query}&type={type}&limit={limit}&offset={offset}` – Пошук пісень, виконавців, альбомів, плейлистів.
 - GET `/stream/{song_id}` – Отримання URL для потокового відтворення.

- GET /recommendations/{user_id}?limit={limit} – Отримання персоналізованих рекомендацій.
- GET /ads/{user_id} – Отримання реклами для користувача.
- GET /users/{id} – Отримати профіль користувача.
- POST /playlists – Створити плейлист.
- GET /songs?artist={name} – Пошук треків за виконавцем.
- POST /subscriptions – Оплата преміуму.
- **Іменування ресурсів:** Використання іменників у множині (/users, /songs).
- **Версіонування:** /api/v1/...
- **HTTP-методи:** GET, POST, PUT, DELETE.
- **Пагінація:** GET /songs?page=2&limit=20.
- **Формат помилок:** HTTP 400 {"error": "Invalid request"}.
- **Статус-коди:** 200 OK, 201 Created, 302 Redirect, 400 Bad Request, 401 Unauthorized, 404 Not Found.

4.1. Open API

```

5. openapi: 3.0.0
info:
  title: Music Streaming Platform API
  version: 1.0.0
  description: API for managing user authentication, streaming
music, searching, recommendations, ads, payments, lyrics, and
analytics.
servers:
  - url: https://api.musicstream.com/v1

paths:
  /users/{id}:
    get:
      summary: Get user profile
      parameters:
        - name: id
          in: path
          required: true
          schema:
            type: string
      responses:
        "200":
          description: User profile retrieved successfully
          content:
            application/json:

```



```
        type: string
title:
  type: string
artist:
  type: string
```

/stream/{song_id}:

```
get:
  summary: Get streaming URL for a song
  parameters:
    - name: song_id
      in: path
      required: true
      schema:
        type: string
  responses:
    "302":
      description: Redirect to streaming URL
      headers:
        Location:
          schema:
            type: string
```

/recommendations/{user_id}:

```
get:
  summary: Get personalized recommendations
  parameters:
    - name: user_id
      in: path
      required: true
      schema:
        type: string
    - name: limit
      in: query
      schema:
        type: integer
        default: 10
  responses:
    "200":
      description: List of recommended songs
      content:
        application/json:
          schema:
            type: object
            properties:
              recommendations:
                type: array
                items:
                  type: object
                  properties:
                    song_id:
                      type: string
```

```
        title:
          type: string
        artist:
          type: string

/ads/{user_id}:
  get:
    summary: Fetch personalized ads
    parameters:
      - name: user_id
        in: path
        required: true
        schema:
          type: string
    responses:
      "200":
        description: Ad information retrieved
        content:
          application/json:
            schema:
              type: object
              properties:
                ad_id:
                  type: string
                ad_url:
                  type: string
                duration:
                  type: integer

/subscriptions:
  post:
    summary: Process premium subscription payment
    requestBody:
      required: true
      content:
        application/json:
          schema:
            type: object
            properties:
              user_id:
                type: string
              payment_method:
                type: string
              amount:
                type: number
    responses:
      "201":
        description: Subscription successful
      "400":
        description: Invalid request

/lyrics/{song_id}:
```

```
get:
  summary: Get lyrics for a song
  parameters:
    - name: song_id
      in: path
      required: true
      schema:
        type: string
  responses:
    "200":
      description: Lyrics retrieved
      content:
        application/json:
          schema:
            type: object
            properties:
              lyrics:
                type: string

/analytics/interactions:
  post:
    summary: Log user interaction
    requestBody:
      required: true
      content:
        application/json:
          schema:
            type: object
            properties:
              user_id:
                type: string
              event:
                type: string
              timestamp:
                type: string
                format: date-time
    responses:
      "200":
        description: Interaction logged
```

Music Streaming Platform API 1.0.0 OAS 3.0

API for managing user authentication, streaming music, searching, recommendations, ads, payments, lyrics, and analytics.

Servers

https://api.musicstream.com/v1

▼

default ^

GET	/users/{id}	Get user profile	▼
GET	/search	Search for songs, artists, albums, or playlists	▼
GET	/stream/{song_id}	Get streaming URL for a song	▼
GET	/recommendations/{user_id}	Get personalized recommendations	▼
GET	/ads/{user_id}	Fetch personalized ads	▼
POST	/subscriptions	Process premium subscription payment	▼
GET	/lyrics/{song_id}	Get lyrics for a song	▼
POST	/analytics/interactions	Log user interaction	▼

default ^

GET /users/{id} Get user profile

Try it out

Parameters

Name	Description
id required string (path)	id

Responses

Code	Description	Links
200	User profile retrieved successfully Media type application/json ▼ Controls Accept header. Example Value Schema <pre>{ "id": "string", "name": "string", "email": "string"}</pre>	No links
404	User not found	No links

GET

/search

Search for songs, artists, albums, or playlists

^

Parameters

Try it out

Name	Description
query required string (query)	query
type string (query)	Available values : song, artist, album, playlist --
limit integer (query)	Default value : 20 20
offset integer (query)	Default value : 0 0

Responses

Code	Description	Links
200	Search results retrieved Media type application/json Controls Accept header. Example Value Schema <pre>{ "results": [{ "type": "string", "id": "string", "title": "string", "artist": "string" }] }</pre>	No links

GET

/stream/{song_id}

Get streaming URL for a song

^

Parameters

Try it out

Name	Description
song_id required string (path)	song_id

Responses

Code	Description	Links						
302	Redirect to streaming URL Headers:<table><thead><tr><th>Name</th><th>Description</th><th>Type</th></tr></thead><tbody><tr><td>Location</td><td></td><td>string</td></tr></tbody></table>	Name	Description	Type	Location		string	No links
Name	Description	Type						
Location		string						

GET

/recommendations/{user_id}

Get personalized recommendations

^

Parameters

Try it out

Name	Description
user_id required string (path)	user_id
limit integer (query)	Default value : 10 10

Responses

Code	Description	Links
200	List of recommended songs Media type application/json Controls Accept header. Example Value Schema <pre>{ "recommendations": [{ "song_id": "string", "title": "string", "artist": "string" }] }</pre>	

 No links |

GET

/ads/{user_id}

Fetch personalized ads

^

Parameters

Try it out

Name	Description
user_id required string (path)	user_id

Responses

Code	Description	Links
200	Ad information retrieved Media type application/json Controls Accept header. Example Value Schema <pre>{ "ad_id": "string", "ad_url": "string", "duration": 0 }</pre>	

 No links |

POST

/subscriptions

Process premium subscription payment

^

Parameters

Try it out

No parameters

Request body

required

application/json

▼

Example Value

Schema

```
{
  "user_id": "string",
  "payment_method": "string",
  "amount": 0
}
```

Responses

Code	Description	Links
201	Subscription successful	No links
400	Invalid request	No links

GET

/lyrics/{song_id}

Get lyrics for a song

^

Parameters

Try it out

Name	Description
song_id required string (path)	song_id

Responses

Code	Description	Links
200	Lyrics retrieved	No links

Media type

application/json

▼

Controls Accept header.

Example Value

Schema

```
{
  "lyrics": "string"
}
```

POST

/analytics/interactions

Log user interaction

^

Parameters

Try it out

No parameters

Request body

required

application/json

▼

Example Value

Schema

```
{
  "user_id": "string",
  "event": "string",
  "timestamp": "2025-03-26T23:12:25.684Z"
}
```

Responses

Code	Description	Links
200	Interaction logged	No links

Рис. 1-9 – Swagger API Docs

Висновок:

У цій роботі було розглянуто процес створення архітектури музичної стрімінгової платформи, враховуючи функціональні та нефункціональні вимоги. Визначено ключові сервіси системи, їх взаємодію, а також розроблено OpenAPI-специфікацію для основних API-ендпоінтів.

Зокрема, сформульовано основні функціональні вимоги, серед яких потокове передавання музики, управління користувачами, персоналізовані рекомендації, інтеграція з платіжними сервісами та зовнішніми API, такими як Musixmatch для відображення текстів пісень. Окрім цього, визначено критично важливі нефункціональні вимоги, що стосуються продуктивності, масштабованості, безпеки та доступності системи.

Архітектурний дизайн побудовано на основі мікросервісного підходу, використовуючи PostgreSQL для збереження метаданих, AWS S3 для зберігання аудіофайлів, Redis для кешування популярного контенту, Kafka для обробки поточкових даних та Kubernetes для автоматичного масштабування.

Для забезпечення ефективного API-інтерфейсу запропоновано набір design guidelines, що включають використання REST, формат JSON, пагінацію, стандартні статус-коди та обробку помилок. Реалізована OpenAPI-документація охоплює сервіси управління користувачами, пошуку контенту, стрімінгу, рекомендацій, реклами, оплати преміум-підписки, інтеграції з сервісами текстів пісень та аналітики взаємодії користувачів.

Запропонована архітектура дозволяє забезпечити високу продуктивність, гнучкість і масштабованість системи, що критично важливо для підтримки понад 100 000 активних користувачів. Отримані результати можуть бути використані як основа для подальшої розробки сервісу, з можливістю розширення функціоналу та інтеграції з додатковими сервісами.